

ECOLE NATIONALE DES SCIENCES APPLIQUEES

LANGAGE C

CHAPITRE 9
LES STRUCTURES

Dalila Raissouni

LES STRUCTURES

- Déclaration d'une structure;
- Utilisation d'une structure;
- Utilisation globale;
- Initialisations;
- Déclaration de types;
- Imbrication de structures;
- Structures comportant des tableaux;
- Tableaux de structures;
- Portée du modèle de structure;
- Transmission d'une structure en argument;
- Transmission de l'adresse d'une structure;
- Transmission d'une structure en valeur de retour.

DÉCLARATION D'UNE STRUCTURE

Définition :

Une structure (struct) est un type de données composite qui permet de regrouper plusieurs variables de types différents sous un même nom logique.

Elle est utilisée pour modéliser des entités réelles complexes (étudiant, bâtiment, date, etc.).

Syntaxe:

```
struct étiquette {
        /* Objet(s) composant(s) la structure. */
    };
```

DÉCLARATION D'UNE STRUCTURE

- **Syntaxe détaillée** :

```
struct NomStructure {
    type1 champ1;
    type2 champ2;
    ...};
```

- **Exemple** :

```
struct Voiture {
  char marque[20];
  char modele[30];
   int annee;
   float prix;
   };
```

UTILISATION D'UNE STRUCTURE

Une structure en C permet de regrouper sous un seul nom plusieurs variables (membres) de types différents pour créer un type de données personnalisé. Ainsi, au lieu de passer plusieurs variables à une fonction:
void afficher(char nom[], char prenom[], int age, float moyenne);

Une seule structure est utilisée:

void afficher(struct candidat c);
Ainsi le code devient :

- plus lisible
- plus propre
- facile à maintenir
- et permet de manipuler des tableaux d'objets.

UTILISATION D'UNE STRUCTURE

En créant une variable etudiant1, on accède à ses membres et on affiche ses informations grace au ' . ':

```
struct Etudiant etudiant1;

    etudiant1.age = 20;
    etudiant1.moyenne = 14.5;

    printf("Âge: %d ans\n", etudiant1.age);
    printf("Moyenne: %.2f/20\n", etudiant1.moyenne);
```

UTILISATION GLOBALE

Une structure peut être déclarée globalement, c'est-à-dire en dehors de toute fonction. Ainsi, elle est accessible globalement, dans les fonctions, les tableaux, et via les pointeurs.

```
#include <stdio.h>
struct Point {     /* Déclaration globale de la structure */
    int x;
    int y; };
void afficherPoint(struct Point p) {  /* Fonction afficherPoint utilise la structure */
    printf("Point (%d , %d)\n", p.x, p.y);}
int main() {
    struct Point p1;
    p1.x = 3;
    p1.y = 5;
    afficherPoint(p1);
    return 0;}
```

INITIALISATIONS

L’initialisation d’une structure en langage C peut se faire de plusieurs manières:

```
struct Etudiant {
char prenom[50];
char nom[50];
int age;
float moyenne;
};
```

Initialisation à la déclaration:

```
struct Etudiant etudiant1 = {"Amine”, “Torres", 20, 15.5};
```

Initialisation partielle:

```
struct Etudiant etudiant2 = {"Narjisse", "El Harrak"};
```

INITIALISATIONS

```
struct Etudiant {
char prenom[50];
char nom[50];
int age;
float moyenne;
};
```

Initialisation désignée (C99) :

```
struct Etudiant etudiant3 = {
.prenom = "Rania",
.nom = "Afailal",
.age=22,
.moyenne = 16.2
};
```

Initialisation à 0 :

```
struct Etudiant etudiant4 = {0};
```

Initialisation membre par membre:

```
struct Etudiant etudiant5;
    etudiant5.age = 21;
    etudiant5.moyenne = 13.7;
```

DÉCLARATION DE TYPES

Pour déclarer un type structure en C, on utilise le mot-clé **struct**, suivi d'une étiquette (nom) et d'une liste de membres, créant ainsi un nouveau type de données complexe ou on utilise **typedef** pour créer un synonyme plus court.

```
struct Voiture {
char marque[20];
char modele[30];
int annee;
};
```

Déclaration:

```
struct Voiture voiture1;
struct Voiture voiture2;
struct Voiture Marque [100];
struct Voiture *p = &voiture1;
```

```
typedef struct {
 char marque[20];
 char modele[30];
 int annee;
} Voiture;
```

Déclaration:

```
 Voiture voiture1;
 Voiture voiture2;
 Voiture Marque[100];
 Voiture *p=&voiture1;
```

DÉCLARATION DE TYPES

```
struct Voiture {
char marque[20];
char modele[30];
int annee;
};
```

Initialisation:

```
strcpy(voiture1.marque, "Toyota");
voiture1.annee = 2020;
struct Voiture voiture2 = {"Peugeot", "208", 2021};
```

Accès via le pointeur:

```
 printf("Accès via pointeur: %s\n", ptr->marque);
```

```
typedef struct {
 char marque[20];
 char modele[30];
 int annee;
} Voiture;
```

Initialisation :

```
strcpy(voiture1.marque, "Toyota");
 voiture1.annee = 2020;
Voiture voiture2 = {"Peugeot", "208", 2021};
```

Accès via le pointeur:

```
 printf("Accès via pointeur: %s\n", ptr->marque);
```

Remarque :

-> est obligatoire pour les pointeurs (ptr->marque équivalent à (*ptr).marque)

IMBRICATION DE STRUCTURES

L'imbrication de structures en langage C consiste à définir une structure à l'intérieur d'une autre structure, permettant de créer des types de données complexes et hiérarchisés avec une meilleure organisation.

Exemple :

```
typedef struct {
    int numero;
    char rue[100];
    char ville[50];
    int code_postal;
} Adresse;

typedef struct {
    int jour;
    int mois;
    int annee;
} Date;
```

```
// Structure Étudiant qui contient d'autres structures
typedef struct {
 char nom[50];
 char prenom[50];
 int numero_etudiant;

 Date date_naissance;

 Adresse adresse;

 float moyenne;
} Etudiant;
```

STRUCTURES COMPORTANT DES TABLEAUX

La syntaxe consiste à déclarer la struct avec des champs (variables) et des **tableaux**, puis à créer une variable de ce type, en accédant aux éléments via l'opérateur **point**.

Exemple :

```
struct Etudiant {
char prenom[50];
char nom[50];
int age;
float moyenne;
};
struct Etudiant e;
    printf("Entrer le nom : ");
    scanf("%49s", e.nom);
    printf("Nom : %s\n", e.nom);
    return 0;
}
```

TABLEAUX DE STRUCTURES

Un tableau de structures est une collection de plusieurs variables de type struct, (chaque élément du tableau étant une structure complète) qui permet de stocker des informations hétérogènes pour plusieurs entités similaires.

Exemple :

```
typedef struct {
  char nom[50];
  char prenom[50];
  int numero_etudiant;
  float moyenne;
} Etudiant;

int main() {

    Etudiant promo[5];
```

PORTÉE DU MODÈLE DE STRUCTURE

La portée du modèle de structure dépend de son emplacement : Si la structure est définie en dehors de toute fonction, elle est accessible partout dans le fichier source ; si elle est définie dans une fonction, elle est locale à cette fonction.

Exemple : **Portée locale**

```
#include <stdio.h>
void afficher() {
    struct Point {    int x;      int y;    };
    struct Point p;
    p.x = 3;
    p.y = 4;
    printf("Point (%d, %d)\n", p.x, p.y);
}
int main() {
    afficher();
    return 0;
}
```

PORTÉE DU MODÈLE DE STRUCTURE

La portée du modèle de structure dépend de son emplacement : Si la structure est définie en dehors de toute fonction, elle est accessible partout dans le fichier source ; si elle est définie dans une fonction, elle est locale à cette fonction.

Exemple : **Portée globale**

```
#include <stdio.h>
struct Point { /* Déclaration globale de la structure */
 int x;
 int y; };
void afficherPoint(struct Point p) { /* Fonction afficherPoint utilise la structure */
 printf("Point (%d , %d)\n", p.x, p.y);}
int main() {
struct Point p1;
 p1.x = 3;
 p1.y = 5;
 afficherPoint(p1);
 return 0;}
```

TRANSMISSION D'UNE STRUCTURE EN ARGUMENT

La transmission d'une structure en argument consiste à passer une variable de type structure à une fonction afin qu'elle puisse accéder à ses champs. Cette transmission peut se faire par valeur (copie de la structure) dans l'**exemple 1** ou par adresse (pointeur vers la structure) dans l'**exemple 2.**

Exemple 1:

```
typedef struct {
char nom[50];
char prenom[50];
int age;
float moyenne;
} Etudiant;

void afficherEtudiantValeur(Etudiant e) {
printf("Nom: %s %s\n", e.prenom, e.nom);
printf("Âge: %d ans\n", e.age);
printf("Moyenne: %.2f/20\n", e.moyenne);
}
```

TRANSMISSION DE L'ADRESSE D'UNE STRUCTURE

La transmission d'adresse d'une structure en C consiste à passer l'adresse mémoire d'une variable structure à une fonction via un pointeur, permettant à la fonction de lire et modifier directement les champs de la structure d'origine, en utilisant l'opérateur flèche (->) pour y accéder.

Exemple 2 :

```
typedef struct {
char nom[50];
char prenom[50];
int age;
float moyenne;
} Etudiant;

void afficherEtudiantPointeur(Etudiant *e) {
    printf("Nom: %s %s\n", e->prenom, e->nom);
    printf("Âge: %d ans\n", e->age);
    printf("Moyenne: %.2f/20\n", e->moyenne);
}
```

TRANSMISSION D'UNE STRUCTURE EN VALEUR DE RETOUR

La transmission d'une structure en valeur de retour en C permet à une fonction de retourner une copie complète de la structure définie, en utilisant **return** suivi du nom de la variable structure, donc la syntaxe implique de déclarer le type de retour de la fonction comme étant la structure elle-même, puis d'utiliser return dans la fonction.

Exemple :

```
typedef struct {
char nom[50];
char prenom[50];
int age;
float moyenne;
} Etudiant;
```

```
Etudiant creerEtudiant() {
 Etudiant e;
printf("\nCréation d'un nouvel étudiant:\n");
printf("Nom: ");
scanf("%s", e.nom);
printf("Prénom: ");
scanf("%s", e.prenom);
printf("Âge: ");
scanf("%d", &e.age);
printf("Moyenne: ");
scanf("%f", &e.moyenne);
return e; }
```

EXERCICE 1

Ecrire un programme en langage C sur la gestion d'un étudiant en définissant une structure Etudiant contenant :

- **un entier id**
- **un nom**
- **une moyenne float**

- **En affectant à cet étudiant l'identifiant =1, le nom: Ali et la moyenne=14,75**

Remarque : Toutes les manières d'initialisation sont considérées justes.

EXERCICE 1

```
#include <stdio.h>
struct Etudiant {
    int id;
    char nom[30];
    float moyenne;
};
int main() {
    struct Etudiant e;
    e.id = 1;
    e.moyenne = 14.75;
//sprintf fonction de stdio.h qui permet d’écrire une chaîne de caractères dans un tableau de char
    snprintf(e.nom, sizeof(e.nom), "Ali");
    printf("ID : %d\n", e.id);
    printf("Nom : %s\n", e.nom);
    printf("Moyenne : %.2f\n", e.moyenne);
    return 0;}
```

EXERCICE 2

Ecrire un programme en C pour gérer une voiture en utilisant une structure Voiture avec typedef et une fonction “affichervoiture”.

La structure doit contenir :

- **une marque (marque) : tableau de caractères de 30**
- **un modèle (modele) : tableau de caractères de 30**
- **une année (annee) : entier**

Dans le programme, créer une voiture avec :

- **marque = "X"**
- **modèle = "Y"**
- **année = Z**

(avec X,Y et Z les valeurs de la voiture que vous désirez)

EXERCICE 2

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char marque[30];
    char modele[30];
    int annee;
} Voiture;
void afficherVoiture(Voiture v) {
    printf("Marque : %s\n", v.marque);
    printf("Modèle : %s\n", v.modele);
    printf("Année : %d\n", v.annee);
}
int main() {
   Voiture v;
    strcpy(v.marque, "X");
    strcpy(v.modele, "Y");
    v.annee = Z;
    afficherVoiture(v);
    return 0; }
```

EXERCICE 2

Ecrire un programme en C pour gérer plusieurs smartphones en utilisant une structure Smartphone avec typedef.

La structure doit contenir :

- **marque (marque) : tableau de 30 caractères**
- **modèle (modele) : tableau de 30 caractères**
- **capacité de stockage (stockage) : entier (en Go)**

Le programme doit :

1. **Permettre à l'utilisateur d'entrer les informations pour 3 smartphones.**
2. **Stocker les informations dans un tableau de structures : telephones[3]**
3. **Afficher tous les smartphones dont la capacité de stockage est supérieure ou égale à 128 Go.**

EXERCICE 3

```
int main() {
    Smartphone telephones[3];
        for(int i = 0; i < 3; i++) {
        printf("Smartphone %d :\n", i+1);
        printf("Marque : ");
        fgets(telephones[i].marque, sizeof(telephones[i].marque), stdin);
        telephones[i].marque[strcspn(telephones[i].marque, "\n")] = '\0';
        printf("Modèle : ");
        fgets(telephones[i].modele, sizeof(telephones[i].modele), stdin);
        telephones[i].modele[strcspn(telephones[i].modele, "\n")] = '\0';
        printf("Capacité de stockage (Go) : ");
        scanf("%d", &telephones[i].stockage);
        getchar(); // vider le buffer
        printf("\n");    }
        for(int i = 0; i < 3; i++) {
        if(telephones[i].stockage >= 128) {
            afficherSmartphone(telephones[i]);       }    }
    return 0;}
```

EXERCICE 3

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char marque[30];
    char modele[30];
    int stockage;
} Smartphone;
void afficherSmartphone(Smartphone s) {
    printf("Marque : %s\n", s.marque);
    printf("Modèle : %s\n", s.modele);
    printf("Stockage : %d Go\n", s.stockage);
  }
int main() {
    Smartphone telephones[3];
    for(int i = 0; i < 3; i++) {
        printf("Smartphone %d :\n", i+1);
        printf("Marque : ");
        fgets(telephones[i].marque, sizeof(telephones[i].marque), stdin);
        telephones[i].marque[strcspn(telephones[i].marque, "\n")] = '\0';
```